

10.AI 学习之路 · 实战篇之「Vibe Coding 实战：从架构到第一个能跑的东西」

来自：帅张和他的朋友们

stormzhang

2026年06月18日 17:06

扫码
查看更

 **系列导航：** 上一篇 [09.技术思维] 帮你列好了一份「照搬就能用」的选型清单。这一篇是实战篇的收官——**Vibe Coding 实战：** 三件套齐了，咱们正式动手，把它做成一个真能跑起来的东西。

其实我们之前的产品设计和架构阶段搞定了之后，基本上就可以直接扔给 AI，让它直接开干了，所以我认为 Vibe Coding 环节是最简单的阶段。

而这阶段的差异就是 Claude Code/Codex 这类 AI Agent的使用熟练程度，以及你用的什么大模型，会导致执行效果有差异。

不过今天还是给大家补一篇 Vibe Coding 环节的教程。

大部分人要做一个产品，比如知识库的时候，第一选择是：

打开 Claude Code，敲了一句：「帮我做一个个人知识库，能抓链接、自动整理、还能搜索。」

然后，它哗哗写了十几个文件、几百行代码。你兴冲冲一跑——报错。改一个错，又冒出三个新错。你想看看到底哪儿出了问题，代码哗啦一堆，互相牵着，从哪查起都没头绪。

这就是大部分 Vibe Coding 经历的常态，**问题不在 AI，在你不会指挥它。**

所以这就是我之所以教大家 Agentic Coding 的重要性。

今天这一篇，我就把「怎么指挥」这件事，掰开揉碎讲给大家。

看完这一篇，你会拿到：

- 一套照着做就能用的 **Agentic Coding 四步法**（喂上下文 / 分步推进 / 每步验证 / 卡住止损）
- 一份「扔链接进去、AI 当场生成笔记」的零代码 workflow 配置（fuxi 同款，复制就能改）
- 一次从空文件夹到「能搜出结果」的完整实录，连报错怎么治都给你看
- 我自己做 fuxi 踩过的几个真坑，帮你少走很多弯路

01 Vibe Coding 不是魔法，是你当包工头

Vibe Coding 是 AI 大神卡帕西 2025 年初造的词，在当下已经演变成「口喷编程」了——你别管代码细节，把活儿交给 AI，你只盯着「做出来的东西对不对」。

听起来很爽。于是很多人理解歪了，以为 Vibe Coding 是：**我说一句话，AI 给我吐一个 App。**——我刚学 AI 的时候也是这么想的。

这里得说一句实话：**Vibe Coding 这个词本身，今天已经过时了。**

这词造出来一年多，行业里也琢磨明白了——真做点东西的「vibe」，根本不是「一句话变 App」，而是 **Agentic Coding**：你当指挥，AI 当能干活的助理，分步派活、每步验证、闭环执行。

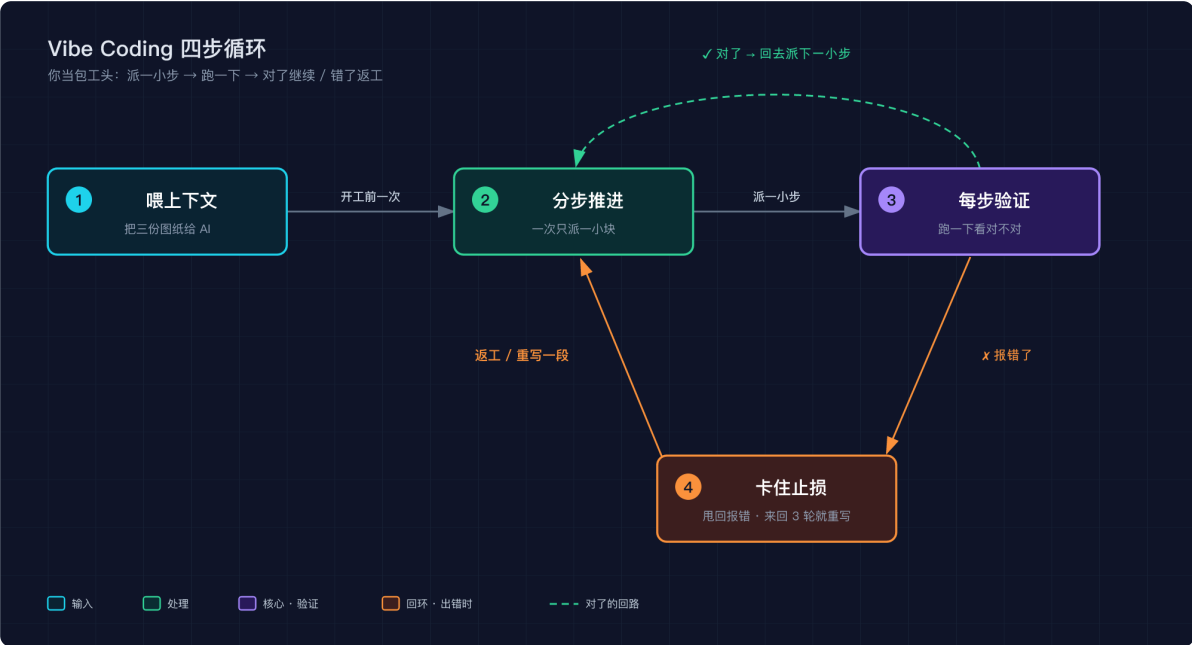
标题写 Vibe Coding，是因为它出圈、好认；但接下来这一篇我教你的，**全是真正能扛住生产的 Agentic Coding**。这俩你别混了——一个是拍脑袋的玩具，一个是真做事的方法。

类比：你是包工头，AI 是施工队。 你不用自己砌墙、布线、刷漆（写代码），但你得干三件事：说清楚要盖成啥样（讲需求）、一层一层验收（做验证）、哪面墙歪了让工人返工（调 bug）。你要是甩一句「给我盖栋楼」就走人，回来保证是一堆废墟。

很多人就是个甩手包工头。

	❌ 甩手掌柜	✅ 称职包工头（你该做的）
怎么开口	一句话「做个知识库」	先喂图纸，再分步派活
进度	让它一口气写完	一次只做一小块
验收	全做完才跑，一跑全崩	每做一步跑一步
出错时	一堆错，不知从哪查	错少、好定位、能返工

记住这张表，因为接下来整篇，我都在教你怎么从左边那列，变成右边那列。具体就是**四步**：



（这张图是这篇的骨架：喂上下文 → 分步推进 → 每步验证 → 卡住止损，是一个不断转的圈，不是一条直线——每做完一小步，就转回来再派下一步。）

这里有个让你松一口气的真相：**整个过程，你不用看懂任何一行代码。**

你是包工头，不是工人。你要会的是「说清楚、会验收、能返工」，不是「会砌墙」。代码长啥样，交给 AI；东西跑没跑通，你说了算。

💡 **一句话总结：**Agentic Coding 不是「说句话变出 App」，是你当包工头——说清需求、分步验收、出错返工，代码细节一行都不用懂。

02 开工第一步：把那三份文档喂给 AI

Agentic Coding 的成败，八成在你敲第一个字之前就定了。

定在哪？上下文——也就是「你有没有先把图纸给施工队」。我第一天那句「做个知识库」，就是甩手包工头让施工队凭空想象，能给我一坨四不像就不错了。

好消息是，咱们前面三篇的功课，现在全派上用场了：

- 07 产品思维 → 你产出过一份「一页纸 MVP」，写清楚做给谁、解决啥
- 08 架构思维 → 你产出过一份架构文档，写清楚「东西怎么走」

这三份，就是你交给施工队的图纸。

怎么喂？很简单——建一个空项目文件夹，把这三份文档丢进去，然后开口第一句不是派活，是让它先读：

```
纯文本

我要做一个个人知识库。这个文件夹里有三份文档：
产品定义、架构、技术选型。
请先把它们读完，用你自己的话复述一遍我要做的是什麼，
确认我们理解一致后，再给我整理一份开工前的设计文档。
再等我派第一个任务，先别写代码。
```

这一句里藏了两个关键动作：**让它复述**（检验它真读懂了没）、**让它先别写代码**（你还没派活，别让它擅自开干）。

我现在每开一个新项目，第一件事都是这个。就这一步，能帮你省掉后面 80% 的「它怎么做的跟我想的完全不一样」。

 **一句话总结：**开口第一句不是「给我写」，是「先读我的图纸、复述一遍对齐」——上下文喂到位，后面才不跑偏。



03 第一个高光：扔个链接，AI 当场给你生成一篇笔记

上面环节 AI 整理了一份精简的，它可以直接开干的产品设计文档，对于新手来说，最好的方式是让 AI 一个一个功能的逐步开干，这便于更好的学习与理解 AI 的工作方式和底层逻辑。

第一个动手的功能，永远挑「入口」——也就是「东西怎么进来」。入口不通，后面存什么、搜什么都是空的。

对 fuxi 来说，入口功能就是 **ingest**：你扔个链接进来，它自动抓取、读懂、整理成一篇笔记。

fuxi 里「抓链接 → 生成笔记」这套，不是一个程序，而是一段**写给 AI 看的规则**。你把规则讲清楚，AI 每次照着做就行。规则写在一个叫 `CLAUDE.md` 的文件里（还记得 04、05 讲的 Skill 和工作流吗？就是它的延续）。

规则长这样，你复制下来改改就能用：

```
纯文本

收到一个链接时，按这个流程处理：

1. 打开链接，把正文从头读完
2. 判断它在讲什麼，提炼成 2-3 句话的摘要 + 3-5 个要点
3. 给它打 3-5 个标签（不够就少打，别硬凑）
4. 存成一个 Markdown 文件，放进 notes/ 文件夹
   文件名用「日期-标题.md」
   文件开头写好：标题、原链接、标签、日期
```

就这么几行大白话。然后你真的扔一个链接进去，AI 当场就给你吐出一篇笔记：

```
markdown

---
title: 某篇讲 AI Agent 的文章
url: https://...
tags: [AI, Agent, 工作流]
date: 2026-06-11
---

## 摘要
这篇文章讲了 Agent 的三个核心能力..... (2-3 句)

## 要点
- 要点一
- 要点二
- 要点三
```

成了。你没写代码，AI 就把「读文章、提炼、打标签、存文件」这一整套全干了——这就是 Agentic Coding 里最反直觉、也最爽的一点：**有些活根本不用写程序，你把要求说清楚，AI 当场就替你干了。**

当然，我也在这儿栽过。第一版规则我写得太死，要求「每篇必须打 5 个标签」。结果有些文章明明俩标签就讲完了，AI 为了凑够 5 个，硬编出一堆不相干的词。后来我改成「3–5 个，不够就少打」，立马正常了。

规则是你给 AI 的需求说明书：写太死，它就僵；写太松，它就飘。这个度，得你来把。

💡 **一句话总结：***fluxi 最核心的「抓链接生成笔记」零代码——一段大白话规则交给 AI 照做就行；规则别写太死，留点余地它才不僵。*

04 该上代码了：让 AI 写一个「能搜」的脚本

笔记攒到几十篇，你就想搜了：「我之前存的那篇讲 Agent 的在哪？」

这时候你当然也能直接问 AI「帮我搜 Agent」。但它每次都得把所有文件从头扫一遍，**又慢又费 token**。这种「规则很明确、要反复干」的机械活，最适合落成一个脚本——写一次，以后一秒出结果。

这一节真要让 AI 写代码了。指挥的核心纪律就一条：**别让它一口气写完，拆成小步，一步一跑。**

别这么说：「写个搜索功能。」（这就是我第一天的死法）

要这么说，一步一步来：

纯文本

第一步：先写一个脚本，能列出 notes/ 里所有笔记的标题。先做这个，做完我跑一下。

→ 跑完你会看到一串文件名往下刷，几十条全在那儿。**对齐认知：哦，它真能读到我所有的笔记。**

跑通了，再派下一步：

纯文本

第二步：现在加一个功能，我给一个关键词，只列出标题里包含这个词的笔记。

→ 跑完输入 `Agent`，列表瞬间缩到剩 3–5 条标题里带 Agent 的。****对齐认知：能搜了。****

再跑通，再下一步：

纯文本

第三步：现在让它不只搜标题，也搜每篇的标签和摘要。

→ 跑完再输入 `Agent`，比上一步多出来几条——标题里没 Agent、但摘要里聊到了 Agent 的那些，也被翻出来了。****对齐认知：能搜深了。****

每加一步，你就在终端跑一下，对了再往下。最后你就得到一个能用的搜索算法脚本「search.py」，不过代码仍然是 AI 写的。

AI 内部搜索的方式其实是它自己写了这个搜索脚本，它实际跑的是类似下面的一个 python 脚本。

纯文本

```
$ python3 search.py Agent
```

找到 3 条：

```
2026-06-11-某篇讲AI-Agent的文章.md
2026-05-28-Agent工作流设计.md
2026-05-10-多Agent协作.md
```

这里再送你一个我踩过的坑。第一版搜索，我贪心，一句话让 AI「做个搜索，支持关键词、标签、日期范围、正则、模糊匹配」。它真给我全写了，参数一大堆。结果呢？我日常 99% 的时候就用关键词那一个，其余全是摆设。后来我砍到只剩「关键词 + 标签」两个，清爽多了。

贪多，是 Agentic Coding 的第一杀手。先做最小的那个能用版本，缺啥再加，永远比一上来求大求全强。这正是 09 反复砸的那句——**优先走轻量方案**。

 **一句话总结：**机械活才让 AI 写脚本，AI 写也得拆小步、一步一跑，代码你不用看懂；先做最小能用版，别一上来求大求全。

05 报错了怎么办：把红字原样甩回去

只要你动手，就一定会报错。没有例外。

报错本身不可怕，可怕的是你看到满屏红字就慌、就觉得「我果然不行」。打住——你是包工头，墙砌歪了是工人返工，不是你的锅。

而你要做的，就一个动作：**把报错原样复制，整段甩回给 AI，说「报这个错，帮我修」。**

给你看段我自己的真实实录。当时写完搜索，我一搜中文关键词，啪，蹦出一串：

纯文本

UnicodeDecodeError: 'utf-8' codec can't decode byte ...

这种错我自己读代码也能修，但 AI 时代，我们一行代码都不碰——能甩回去的，就别自己花时间。直接复制，甩回给 Claude：「报这个错，帮我修。」它说是文件编码的问题，改一行就好。我一跑——

第二轮：又报错，说某个文件读不到。甩回去，它说有篇笔记文件名带了特殊符号，加了一行容错。

第三轮：再跑，这次能搜了但中文标签匹配不上。甩回去，它说要做 Unicode 归一化，又改一行。

第四轮：跑通。

前后没超过十分钟，我全程一行 trace 都没自己去翻。

说句实话：Agentic Coding 真正的省时点不在「不会调 bug」，在「你的时间比 trace 值钱」。这条对你也成立——你不需要懂这些错是啥，照样能调通。

调 bug 你只要守住三条纪律：

❌ 新手（越调越乱）		✅ 老手（越调越顺）
怎么报错	自己用话转述「它好像打不开文件」	报错原样复制甩回去，一字不改
改几处	让它一次改五个地方	一次只改一处，改完立刻跑
卡住时	同一个错死磕半天，越改越乱	来回 3 轮还不好，就让它「这段重写」

第三条我得多说一句。有一次一个 bug，我和 Claude 缠了快一个钟头，改了又改，越改越乱，文件都快不成样子了。最后我一咬牙：「别修了，把这个文件整个删了，按原来的需求重写一遍。」5 分钟，搞定。

有时候，重来比修快。别因为「已经改了这么多」就舍不得推倒——沉没成本不值钱，跑通才值钱。

这条「3 轮不行就重写」，就是四步法里最后一步——**卡住止损**。



 **一句话总结：**报错原样甩回去、一次只改一处、卡住超过 3 轮就让它重写（这就是「卡住止损」）——你全程不用看懂任何一个错。

06 一条贯穿全篇的规律：语义活让 AI 当场干，机械活让 AI 写脚本

回头看你今天做的两件事，藏着 fuxi 的全部秘密：

- ingest（抓链接生成笔记）——没写代码，一段规则交给 AI 当场干
- search（搜索）——AI 写了脚本，落成一秒出结果的程序

凭啥一个写代码、一个不写？区别就一条：

	语义活（交给 AI 当场干）	机械活（落成脚本）
特点	要理解、要判断	规则明确、反复重复
例子	读懂文章、提炼观点、打标签、判断哪两篇有关联	扫文件、关键词匹配、生成索引、统计标签
怎么实现	写进 <code>CLAUDE.md</code> 的大白话规则	让 AI 写一个脚本
为啥这么分	这些活只有「会理解」的 AI 干得了	这些活机械、高频，脚本快又稳、还省 token

这就是 09 结尾那句「机械归脚本、语义归 Claude」落到地上的样子。

往大了说，这揭示了 AI 时代做产品的一个真相：你的大部分「开发」，根本不是在写代码，而是在写清楚需求。你今天写得最多的，不是 Python，是那段大白话规则——这跟 04 的 Skill、05 的工作流是同一条线。

而「把需求说清楚」这件事，恰恰不需要你是程序员。它要的是产品思维、是想清楚——就是 06–09 那四篇干的事，全都接上了。

 **一句话总结：**语义活（理解、判断）交给 AI 写进规则，机械活（重复、明确）才让 AI 落成脚本——AI 时代的「开发」，大半是把需求说清楚，不是敲代码。

07 动手环节：今天就做出你的第一个小功能

光看不练，三天就忘。今天就动手，做一个你自己的、最小的能跑的东西。

前提：你得有 Claude Code 或 Codex 其中一个（没装的话，去官网搜「Claude Code 安装」或「Codex 安装」，照着走十分钟搞定，这里不展开）。

然后三步：

- 1 建个空文件夹，把你 07、08、09 那三份文档丢进去，让 AI 先读完、复述一遍对齐
- 2 挑一个最小的功能——别挑大的，就挑「扔个链接、生成一篇笔记」这种，写 5 行大白话规则让 AI 干
- 3 扔一个真实输入进去，看它跑出啥；报错就原样复制甩回去，来回几轮治通

就这么简单。记住唯一的纪律：**选最小的那个，先跑通一个**。跑通一个最小的，比憋一个大的强一百倍——因为跑通的那一下，你会真正相信「原来我也能做出东西」，这个信心，比任何教程都重要。

 **一句话总结：**建空文件夹 → 喂三份文档对齐 → 挑最小功能写 5 行规则 → 扔真实输入治通，今天就跑出你的第一个东西。

08 小结：实战篇收官，你已经做出了一个属于自己的东西

这一篇的核心，就是一套 Agentic Coding 四步法：

喂上下文 → 分步推进 → 每步验证 → 卡住止损。

外加一条分工铁律：**语义活让 AI 当场干，机械活让 AI 落成脚本**。

到这里，实战篇正式收官。我们一起从头走了一遍：

篇	你拿到了什么
06 为什么做	想清楚为啥要自己搭一个知识库
07 产品思维	一份一页纸 MVP：做给谁、解决啥
08 架构思维	一份架构文档：东西怎么走
09 技术思维	一份选型清单：用啥工具
10 Agentic Coding	把上面全部落地，做出一个能跑的东西

从「为什么做」到「做出来」，五篇走完，你手上已经有一个能跑的、属于你自己的知识库雏形了。这不是看会的，是做出来的——这俩差着十万八千里。

如果你跟着动手了，哪怕只跑通了「扔链接生成一篇笔记」这一个最小功能，你也已经跨过了多数人一直没跨过的那道坎：从「我不会编程」到「我用 AI 做出了一个真东西」。

AI 知识库实战篇到这个阶段，每个人如果跟着实践，应该基本都拿到了一个本地最小可用知识库产品。

因为这是属于每个人的专属知识库，而且每个人的需求和场景肯定都不一样，一开始不知道优化改进方向没关系，先每天用起来，只有你自己常用，你才能知道你自己的需求是什么，然后借助 AI，不断去优化细节，逐步扩展功能就好。

本系列持续更新，建议收藏。